

2. CLI CHATBOT

Implemented clear command and persona flag

```
PS C:\Users\Mashal Zahra\Downloads\projects\clibot> python -m bot.cli
Chatting with gemini (gemini-2.5-flash). Type /help for commands.
you> hi
CALLING GEMINI...

gemini> Hello!
you> /help

Commands:
/clear          Clear conversation history
/save <file>    Save conversation history to a JSON file (default: conversation.json)
/load <file>    Load conversation history from a JSON file
/quit          Exit

you> /save
Saved to conversation.json
you> /load
Loaded from conversation.json
you> /clear
Conversation cleared.
you> /quit
PS C:\Users\Mashal Zahra\Downloads\projects\clibot> python -m bot.cli --persona "You are a Python tutor"
```



```
CLIBOT conversation.json > ...
1  [
2    {
3      "role": "system",
4      "content": "You are a helpful, concise assistant."
5    },
6    {
7      "role": "user",
8      "content": "hi"
9    },
10   {
11     "role": "assistant",
12     "content": "Hello!"
13   }
14 ]
```

```

you> explain blackhole
CALLING GEMINI...

gemini> Okay, let's dive into one of the most mind-bending concepts in the universe: a black hole!

Imagine something so incredibly dense that its gravity is inescapable, not even for light itself. That's the essence of a black hole.

Here's a breakdown:

### The Core Idea: Extreme Gravity

1. Gravity: We're all familiar with gravity - it's what keeps us on Earth, and it holds planets in orbit around the Sun. The strength of gravity depends on two things:
    * The mass of an object (how much "stuff" it has).
    * The distance from the object.

2. Escape Velocity: To leave Earth, a rocket needs to reach a certain speed (about 11.2 kilometers per second) to overcome Earth's gravity. This is called the escape velocity. If you throw a ball slower than that, it comes back down.

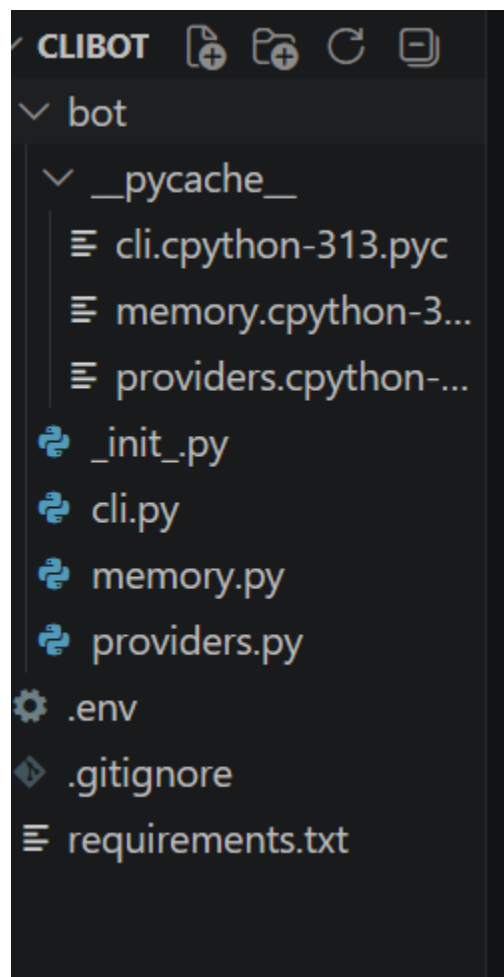
3. Black Hole's Gravity: For a black hole, the object's mass is compressed into such an incredibly small space that its gravity becomes astronomically powerful. So powerful, in fact, that the escape velocity from its surface would be greater than the speed of light (299,792,458 meters per second).

4. Nothing Escapes: Since nothing in the universe can travel faster than the speed of light, this means that once you cross a certain boundary around a black hole, you can never escape - not even light itself. That's why they are "black" - they don't reflect or emit any light.

### The Anatomy of a Black Hole

* Event Horizon: This is the most famous part. It's not a physical surface you can touch, but rather a theoretical boundary. Think of it as the "point of no return." Once something crosses the Event Horizon, it's irretrievably pulled into the black hole. The universe outside can't see what happens inside.
* Singularity: At the very center of a black hole lies the singularity. This is where all the mass of the black hole is thought to be compressed into an infinitely small, infinitely dense point. Our current laws of physics (specifically, Einstein's General Relativity) begin to break down when trying to describe what exactly happens at this point.

```



bot->memory

```
"""
Sliding-window conversation memory with token-aware trimming.
Uses tiktoken's cl100k_base encoding as a token-count approximation –
not perfectly exact for Gemini, but close enough to budget against.
"""

import json
import tiktoken

_ENCODER = tiktoken.get_encoding("cl100k_base")

class ConversationManager:
    def __init__(self, system_prompt: str | None = None, max_tokens: int = 3000):
        self.max_tokens = max_tokens
        self.messages: list[dict] = []
        if system_prompt:
            self.messages.append({"role": "system", "content": system_prompt})

    def _count(self, text: str) -> int:
        return len(_ENCODER.encode(text))

    def total_tokens(self) -> int:
        return sum(self._count(m["content"]) for m in self.messages)

    def add(self, role: str, content: str) -> None:
        self.messages.append({"role": role, "content": content})
        self._trim()

    def _trim(self) -> None:
        """
        Drop the oldest non-system message first, repeatedly, until back
        under budget. If a single message is larger than the whole budget
        this won't fully save you – truncate giant inputs before they
        arrive.
        """
        while self.total_tokens() > self.max_tokens and len(self.messages) > 1:
```

```

        drop_index = 1 if self.messages[0]["role"] == "system" else 0
        if len(self.messages) <= drop_index + 1:
            break
        del self.messages[drop_index]

    def save(self, path: str) -> None:
        with open(path, "w") as f:
            json.dump(self.messages, f, indent=2)

    def load(self, path: str) -> None:
        with open(path) as f:
            self.messages = json.load(f)

    def clear(self) -> None:
        system=[]

        if self.messages and self.messages[0]["role"]=="system":
            system.append(self.messages[0])

        self.messages=system

```

This file handles the chatbot's conversation memory. Its job is to store messages, count approximate tokens, and automatically remove old messages if the conversation becomes too long.

To estimate token count this function is used: `_ENCODER = tiktoken.get_encoding("cl100k_base")`

class `ConversationManager` is responsible for managing history.

To initialize memory: `__init__(system_prompt, max_tokens)`

`save(path)` - saves conversation history into a JSON file

`clear()`- is the custom function added to clear conversation w/o restarting chatbot

bot->cli

```
"""
```

Multi-provider CLI chatbot with sliding-window memory and live cost tracking.

Run with: `python -m chatbot.cli`

```
"""
import argparse
from dotenv import load_dotenv
load_dotenv() # reads .env file so GEMINI_API_KEY is available
from bot.memory import ConversationManager
from bot.providers import call_model

DEFAULT_MODELS = {
    "gemini": "gemini-2.0-flash",
}

HELP_TEXT = """
Commands:
    /clear                Clear conversation history
    /save <file>          Save conversation history to a JSON file (default:
conversation.json)
    /load <file>          Load conversation history from a JSON file
    /quit                 Exit
"""

def run(provider: str, system_prompt: str, max_tokens: int,
input_fn=input, print_fn=print):
    """
    Core REPL loop. input_fn/print_fn are injectable so tests can run
    without touching real stdin/stdout.
    """
    model = DEFAULT_MODELS[provider]
    memory = ConversationManager(system_prompt=system_prompt,
max_tokens=max_tokens)

    print_fn(f"Chatting with {provider} ({model}). Type /help for
commands.")

    while True:
        try:
```

```

        user_input = input_fn("you> ").strip()
    except (EOFError, KeyboardInterrupt):
        print_fn("\nbye.")
        return

    if not user_input:
        continue
    if user_input == "/quit":
        return
    if user_input == "/help":
        print_fn(HELP_TEXT)
        continue
    if user_input == "/clear":
        memory.clear()
        print_fn("Conversation cleared.")
        continue
    if user_input.startswith("/save"):
        parts = user_input.split(maxsplit=1)
        path = parts[1] if len(parts) == 2 else "conversation.json"
        memory.save(path)
        print_fn(f"Saved to {path}")
        continue
    if user_input.startswith("/load"):
        parts = user_input.split(maxsplit=1)
        path = parts[1] if len(parts) == 2 else "conversation.json"
        memory.load(path)
        print_fn(f"Loaded from {path}")
        continue

    memory.add("user", user_input)
    reply, usage = call_model(provider, model, memory.messages)
    memory.add("assistant", reply)
    print_fn(f"\n{provider}> {reply}")

def main():
    parser = argparse.ArgumentParser(description="Gemini CLI chatbot with
memory")

```

```

    parser.add_argument("--provider", default="gemini",
choices=DEFAULT_MODELS.keys())
    parser.add_argument("--persona", default="You are a helpful, concise
assistant.")
    parser.add_argument(
        "--max-tokens", type=int, default=3000,
        help="Sliding-window token budget before old messages get dropped"
    )
    args = parser.parse_args()
    run(args.provider, args.persona, args.max_tokens)

if __name__ == "__main__":
    main()

```

This is main entry point of the chatbot project. It creates the command-line interface (CLI), receives user input, manages commands, and connects memory with the Gemini model.

The chatbot is started by typing: `python -m bot.cli`.

HELP_TEXT contains all available commands which are shown when user types /help.

`run(provider, system_prompt, max_tokens)`- is the main chatbot loop and most important function.

Inside while true, messages are continuously accepted

`main()` is responsible for parsing setup arguments.

bot->providers

```

"""
Unified call_model() - Gemini only for now.
To add OpenAI or Anthropic later, add their if-blocks here and nowhere
else.
"""
import os
from google import genai

```

```

_gemini_client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY",
""))

def call_model(provider: str, model: str, messages: list[dict]) ->
tuple[str, dict]:
    """
    messages: list of {"role": "system"|"user"|"assistant", "content":
str}
    Returns (response_text, usage) where usage has
    "input_tokens"/"output_tokens".
    """
    if provider == "gemini":
        system_msg = next((m["content"] for m in messages if m["role"] ==
"system"), None)
        convo = [m for m in messages if m["role"] != "system"]

        # Gemini uses "user"/"model" roles, not "user"/"assistant"
        contents = [
            {
                "role": "user" if m["role"] == "user" else "model",
                "parts": [{"text": m["content"]}],
            }
            for m in convo
        ]

        config = {"system_instruction": system_msg} if system_msg else
None

    print("CALLING GEMINI...")
    resp = _gemini_client.models.generate_content(
        model=model,
        contents=contents,
        config=config,
    )

    text = resp.text
    usage = {
        "input_tokens": resp.usage_metadata.prompt_token_count,

```



```
        "output_tokens": resp.usage_metadata.candidates_token_count,
    }
    return text, usage

    raise ValueError(f"Unknown provider: {provider!r}. Currently only
'gemini' is supported.")
```

This file is responsible for connecting the chatbot with the language model API. It acts as a separate provider layer so if another model is used later, only this file needs modification instead of changing the whole project.

The function `genai.CClient()` creates connection with gemini API.

The main function is: `call_model(provider, model, messages)`.

For the persona flag: `system_msg = next(...)` is used, this instruction affects future replies.

Requirements:

```
1  google-genai
2  tiktoken
3  python-dotenv
4
5  pip install google-genai tiktoken python-dotenv
6
```

Challenges Implemented and Output Differences

In both projects, some small changes and features were added to make the programs work better. In Project 1, validation was added for user input and prompt instructions were written in a safer way so wrong, too long, or unrelated inputs do not affect the output much. Because of this, the generated cover letters became more focused and stayed closer to the required length and tone.

In Project 2), /clear command was added so conversation can start fresh without closing the program, and --persona option was added to change chatbot behaviour. Due to these changes, outputs became different because after clearing memory old chats are removed, and using different personas gives replies in different styles. Screenshots are attached to show implementation and output differences.